LeetCode Practice

7 Medium: JavaScript

Question, examples, then the six-phase interview approach with the brute force in phase 2

A split copy of the full All Questions book; every feature and link works the same

7
MEDIUM

Patterns (in order)

1. JavaScript (7)

How to use

- Tap any title in the Contents to jump to it. A topic name opens that topic's own divider page.
- Every page has < Prev | Contents | Next > at the top.
- Prev and Next move one question at a time; Contents returns here. Each topic starts on its own page.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

7 questions | tap a title to jump to it

Medium questions (7) Go >

JavaScript (7) Go >

M 1. Debounce LC 2627

M 2. JSON Deep Equal LC 2628

M 3. Memoize LC 2630

M 4. Convert Object to JSON String LC 2633

M 5. Promise Pool LC 2636

M 6. Array of Objects to Matrix LC 2675

M 7. Differences Between Two Objects LC 2700

Medium questions (7)

MEDIUM | TOPIC 1 OF 1

JavaScript

7 Medium questions start on the next page

Before this: this is the first topic
After this: this is the last topic

IN THIS TOPIC

1. Debounce LC 2627
2. JSON Deep Equal LC 2628
3. Memoize LC 2630
4. Convert Object to JSON String LC 2633
5. Promise Pool LC 2636
6. Array of Objects to Matrix LC 2675
7. Differences Between Two Objects LC 2700

MEDIUM | JavaScript | LeetCode #2627 | 1 of 7 | 1/7

Debounce

QUESTION

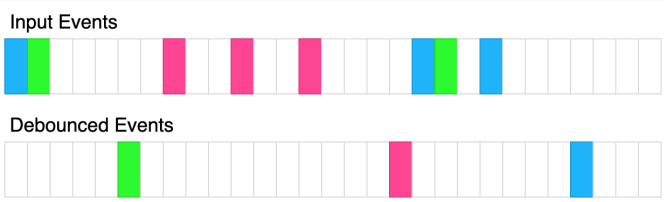
Given a function fn and a time in milliseconds t,
return a debounced version of that function.

A debounced function is a function whose execution is delayed by t milliseconds and whose execution is cancelled if it is called again within that window of time. The debounced function should also receive the passed parameters.

For example, let's say t = 50ms, and the function was called at 30ms, 60ms, and 100ms.

The first 2 function calls would be cancelled, and the 3rd function call would be executed at 150ms.

If instead t = 35ms, The 1st call would be cancelled, the 2nd would be executed at 95ms, and the 3rd would be executed at 135ms.



The above diagram shows how debounce will transform events. Each rectangle represents 100ms and the debounce time is 400ms. Each color represents a different set of inputs.

Please solve it without using lodash's _.debounce() function.

Example 1

Input	`t = 50`
	`calls = [`
	`{"t": 50, inputs: [1]},`
	`{"t": 75, inputs: [2]}`
	`]`

Output	`[{"t": 125, inputs: [2]}]`
Explanation	*let start = Date.now();* *function log(...inputs) {* *console.log([Date.now() - start, inputs])* *}* *const dlog = debounce(log, 50);* *setTimeout(() => dlog(1), 50);* *setTimeout(() => dlog(2), 75);* *The 1st call is cancelled by the 2nd call because the 2nd call occurred before 100ms* *The 2nd call is delayed by 50ms and executed at 125ms. The inputs were (2).*

QUESTION

Example 2

Input	`t = 20`
	`calls = [`
	`{"t": 50, inputs: [1]},`
	`{"t": 100, inputs: [2]}`
	`]`

Output	`[{"t": 70, inputs: [1]}, {"t": 120, inputs: [2]}]`
Explanation	*The 1st call is delayed until 70ms. The inputs were (1).* *The 2nd call is delayed until 120ms. The inputs were (2).*

Example 3

Input	`t = 150`
	`calls = [`
	`{"t": 50, inputs: [1, 2]},`
	`{"t": 300, inputs: [3, 4]},`
	`{"t": 300, inputs: [5, 6]}`
	`]`

Output	`[{"t": 200, inputs: [1,2]},` `{"t": 450, inputs: [5, 6]}]`
Explanation	*The 1st call is delayed by 150ms and ran at 200ms. The inputs were (1, 2).* *The 2nd call is cancelled by the 3rd call* *The 3rd call is delayed by 150ms and ran at 450ms. The inputs were (5, 6).*

CONSTRAINTS

- 0 <= t <= 1000
- 1 <= calls.length <= 10
- 0 <= calls[i].t <= 1000
- 0 <= calls[i].inputs.length <= 10

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

`debounce(fn, t): delay fn's execution by t ms. Each new call resets the timer.`

Only the most recent call fires — earlier pending calls are cancelled. Right?"

"A few quick questions: t can be 0? Yes — fires in the next microtask. No calls = no fires."

"Let me walk: calls at 0ms, 30ms, 60ms with t=50ms -> fn fires once at 110ms ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

PHASE 2 BRUTE FORCE

```
keep a list of timer ids
each call:
    clear every timer in the list one by one,
        then start a new timer
    the function runs when the last timer fires
```

TIME	SPACE
O(c) per call	O(c)

WHY IT IS SLOW

Clears all earlier timers one by one. Keep a single timer id: on each call clearTimeout(timer) and set a new one, O(1) per call and O(1) space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Closure returning delayed executor — O(1) setup. Walk-through:"

debounce(fn, t) returns a new function that:

Cancels any pending timer: clearTimeout(timer).

Sets new timer: timer = setTimeout(() => fn(...args), t).

On each call: if called again within t ms, the previous timer is cancelled.

Only the LAST call within any t-ms window actually executes fn.

WHY clearTimeout before setTimeout: ensures only the most recent call fires.

Use case: search box — only fires after user STOPS typing for t ms.

Time: O(1) per call (timer ops are O(1)). Space: O(1).

PHASE 4 CLEAN CODE

Timer-based: cancel+restart a threading.Timer on each call.

O(1) per invocation. No background polling — event-driven.

```
var debounce = function(fn, t) {
    let timer;
    return function(...args) {
        clearTimeout(timer);
     // a new call cancels the pending one
        timer = setTimeout(() => fn(...args), t);
    };
};
```

PHASE 5 TEST & DEBUG

"Let me trace debounce(fn, t=50ms) with calls at t=0,30,60ms."

State: timerId=null, fn fires once after 50ms silence

t=0ms: call debounced() -> clearTimeout(null) no-op -> timerId=setTimeout(fn,50). timer fires at t=50ms

t=30ms: call debounced() -> clearTimeout(timer50) CANCELLED -> timerId=setTimeout(fn,50). fires at t=80ms

PHASE 5 TEST & DEBUG

t=60ms: call debounced() -> clearTimeout(timer80) CANCELLED -> timerId=setTimeout(fn,50). fires at t=110ms

t=110ms: fn() executes ok. No earlier calls fired.

Trace single call at t=0: clearTimeout(null) -> setTimeout(fn,50) -> fn fires at t=50ms ok

WHY reset timer: debounce means 'only fire after t ms of silence'; each new call restarts clock.

Edge: no calls -> fn never fires. Rapid calls -> fn fires once after last call + t ms.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(1) per call	O(1) one timer reference

Debounce: wait until activity stops. Throttle: fire at most once per interval.

Follow-up: leading debounce — fire immediately then ignore for t ms.

Follow-up: Throttle (#2676) — use setInterval or timestamp-based approach.

MEDIUM | JavaScript | LeetCode #2628 | 2 of 7 | 2/7

JSON Deep Equal

QUESTION

Given two values o1 and o2, return a boolean value indicating whether two values, o1 and o2, are deeply equal.

For two values to be deeply equal, the following conditions must be met:

CONSTRAINTS

- If both values are primitive types, they are deeply equal if they pass the === equality check.
- If both values are arrays, they are deeply equal if they have the same elements in the same order, and each element is also deeply equal according to these conditions.
- If both values are objects, they are deeply equal if they have the same keys, and the associated values for each key are also deeply equal according to these conditions.

You may assume both values are the output of JSON.parse. In other words, they are valid JSON.

Please solve it without using lodash's _.isEqual() function

Example 1

Input	`o1 = {"x":1,"y":2}, o2 = {"x":1,"y":2}`
Output	`true`
Explanation	*The keys and values match exactly.*

Example 2

Input	`o1 = {"y":2,"x":1}, o2 = {"x":1,"y":2}`
Output	`true`
Explanation	*Although the keys are in a different order, they still match exactly.*

Example 3

Input	`o1 = {"x":null,"L":[1,2,3]}, o2 = {"x":null,"L":["1","2","3"]}`
Output	`false`
Explanation	*The array of numbers is different from the array of strings.*

Example 4

Input	`o1 = true, o2 = false`
Output	`false`
Explanation	*true !== false*

- $1 <= JSON.stringify(o1).length <= 10^5$
- $1 <= JSON.stringify(o2).length <= 10^5$
- maxNestingDepth <= 1000

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Deep equality for JSON values: same primitive value (===), same array structure,

same object keys and values recursively. Keys in different order are still equal. Right?"

"A few quick questions: null === null is true. null !== {}? Yes."

"Let me walk: areDeeplyEqual({a:1},{a:1}) -> true. areDeeplyEqual([1],[2]) -> false ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
return JSON.stringify(o1) === JSON.stringify(o2)
```

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

Fast, but wrong: key order matters to stringify, and undefined and NaN or functions behave differently. A recursive comparison (same type, same keys regardless of order, deep-equal values, arrays by index) is O(n) with O(depth) space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Recursive deep equality check — O(n) where n = total nested values. Walk-through:"

If both primitives: return o1 === o2.

If types differ: return false.

If arrays: check same length, then recursively check each index.

If objects: check same keys, then recursively check each value.

Base cases: null, undefined, primitives — handle before object check.

WHY recursive: nested structures require checking all levels.

WHY check keys symmetrically: o1 might have keys o2 doesn't (or vice versa) -> both must match.

Time: O(n) total nodes. Space: O(depth) recursion.

PHASE 4 CLEAN CODE

Recursive type dispatch — O(n) single pass, no serialization.

```
var areDeeplyEqual = function(o1, o2) {
    if (o1 === o2) return true;
    if (o1 === null || o2 === null || typeof o1 !== 'objec
t' || typeof o2 !== 'object') return false;
    if (Array.isArray(o1) !== Array.isArray(o2)) return fa
lse;
    const keys1 = Object.keys(o1);
    if (keys1.length !== Object.keys(o2).length) return fa
```

```
lse;
    for (const key of keys1) {
        if (!(key in o2) || !areDeeplyEqual(o1[key], o2[ke
y])) return false;
    }
    return true;
};
```

PHASE 5 TEST & DEBUG

"Let me trace areDeeplyEqual({y:2,x:1},{x:1,y:2})."

typeof both = 'object', not null, not Array

keys(o1)=['y','x'], keys(o2)=['x','y']. Sort both -> ['x','y']==['x','y'] ok (same key sets)

key 'x': recurse areDeeplyEqual(1,1): typeof='number' -> 1===1 -> true ok

key 'y': recurse areDeeplyEqual(2,2): typeof='number' -> 2===2 -> true ok

All keys equal -> return true ok

Trace areDeeplyEqual(true,false): typeof='boolean' -> true===false -> false ok

Trace areDeeplyEqual([1,2],[1,3]): isArray both ok. len 2==2. idx0: 1===1 ok. idx1: 2!=3 -> false ok

WHY sort keys: object key order is insertion-dependent; sorting ensures order-independent comparison.

Null check before typeof: typeof null === 'object' so explicit null guard needed first.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(d)** depth

Follow-up: handle circular references — track visited pairs with WeakMap.

Follow-up: Differences (#2700) — return diff object instead of bool.

MEDIUM | JavaScript | LeetCode #2630 | 3 of 7 | 3/7

Memoize

QUESTION

Given a function fn, return a memoized version of that function.

A memoized function is a function that will never be called twice with the same inputs. Instead it will return a cached value.

fn can be any function and there are no constraints on what type of values it accepts. Inputs are considered identical if they are === to each other.

Example 1

Input	`getInputs = () => [[2,2],[2,2],[1,2]]` `fn = function (a, b) { return a + b; }`
Output	`[{"val":4,"calls":1},{"val":4,"calls":1},{"val":3,"calls":2}]`
Explanation	*const inputs = getInputs();* *const memoized = memoize(fn);* *for (const arr of inputs) {* *memoized(...arr);* *}* *For the inputs of (2, 2): 2 + 2 = 4, and it required a call to fn().* *For the inputs of (2, 2): 2 + 2 = 4, but those inputs were seen before so no call to fn() was required.* *For the inputs of (1, 2): 1 + 2 = 3, and it required another call to fn() for a total of 2.*

Example 2

Input	`getInputs = () => [[{},{}],[{},{}],[{},{}]]` `fn = function (a, b) { return ({...a, ...b}); }`
Output	`[{"val":{},"calls":1},{"val":{},"calls":2},{"val":{},"calls":3}]`
Explanation	*Merging two empty objects will always result in an empty object. It may seem like there should only be 1 call to fn() because of cache-hits, however none of those objects are === to each other.*

QUESTION

Example 3

Input	`getInputs = () => { const o = {}; return [[o,o],[o,o],[o,o]]; }` `fn = function (a, b) { return ({...a, ...b}); }`
Output	`[{"val":{},"calls":1},{"val":{},"calls":1},{"val":{},"calls":1}]`
Explanation	*Merging two empty objects will always result in an empty object. The 2nd and 3rd third function calls result in a cache-hit. This is because every object passed in is identical.*

CONSTRAINTS

- 1 <= inputs.length <= 10^5
- 0 <= inputs.flat().length <= 10^5
- inputs[i][j] != NaN

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

`memoize(fn): return a cached version that skips fn when same inputs are seen.`

Inputs compared with === (object references, not structure). Right?"

"A few quick questions: {} === {} is false in JS — two empty objects are different? Yes."

"Let me walk: memoized(2,3) first call -> computes. memoized(2,3) again -> cached ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
def memoized(...args):
    call fn(...args) every time
```

TIME	SPACE
O(cost of fn) per call	**O(1)**

WHY IT IS SLOW

Repeats the work for repeated inputs, which is what memoisation exists to avoid. Store results in a map keyed by the arguments (for example a JSON string) and return the cached value on a repeat.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the linear scan for matching args.

Use a Map with JSON.stringify(args) as key for O(1) lookup.

Note: JSON.stringify({}) === JSON.stringify({}) so structurally equal objects share cache.

For reference-equality semantics, a trie or WeakMap-based structure is needed.

Time: O(key_len) per call. Space: O(unique calls).

PHASE 4 CLEAN CODE

Hash map keyed by args tuple — O(1) average lookup.

```
function memoize(fn) {
    const root = new Map();
     // a trie keyed by each argument (compared with ===)
    const RESULT = Symbol('result');
    return function(...args) {
        let node = root;
        for (const arg of args) {
            if (!node.has(arg)) node.set(arg, new Map());
            node = node.get(arg);
        }
        if (node.has(RESULT)) return node.get(RESULT);
        const result = fn(...args);
        node.set(RESULT, result);
        return result;
    };
}
```

PHASE 5 TEST & DEBUG

"Let me trace the logic and highlight the key state updates."

fn=sum. memoized(2,3): key='[2,3]', miss -> compute 5, cache. return 5.

`memoized(2,3): key='[2,3]', hit -> return 5 (fn not called) ok`

`memoized({},{}) x3 (same objects): key='[{},{}]' each time -> cached after first ok`

WHY JSON.stringify key: encodes all args to a string; same primitive inputs always produce same key.

PHASE 6 COMPLEXITY & FOLLOW-UP

First call O(fn). Cache hit O(key_len). Space O(unique key count).

Follow-up: LRU-bounded memoize — evict oldest when cache exceeds size limit.

Follow-up: async memoize — cache the Promise itself so concurrent calls share one fetch.

MEDIUM | JavaScript | LeetCode #2633 | 4 of 7 | 4/7

Convert Object to JSON String

QUESTION

Given a value, return a valid JSON string of that value. The value can be a string, number, array, object, boolean, or null. The returned string should not include extra spaces. The order of keys should be the same as the order returned by Object.keys().

Please solve it without using the built-in JSON.stringify method.

Example 1

Input	`object = {"y":1,"x":2}`
Output	`{"y":1,"x":2}`
Explanation	*Return the JSON representation. Note that the order of keys should be the same as the order returned by Object.keys().*

Example 2

Input	`object = {"a":"str","b":-12,"c":true,"d":null}`
Output	`{"a":"str","b":-12,"c":true,"d":null}`
Explanation	*The primitives of JSON are strings, numbers, booleans, and null.*

Example 3

Input	`object = {"key":{"a":1,"b":[{},null,"Hello"]}}`
Output	`{"key":{"a":1,"b":[{},null,"Hello"]}}`
Explanation	*Objects and arrays can include other objects and arrays.*

Example 4

Input	`object = true`
Output	`true`
Explanation	*Primitive types are valid inputs.*

CONSTRAINTS

- value is a valid JSON value
- 1 <= JSON.stringify(object).length <= 10^5
- maxNestingLevel <= 1000
- all strings contain only alphanumeric characters

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Implement JSON.stringify: handle null, bool, number, string, array, object.

No extra spaces. Keys in Object.keys() order. No eval. Right?"

"A few quick questions: undefined values in arrays become null? No — constraints say valid JSON only."

"Let me walk: jsonStringify({a:2,b:[1,2,3]}) -> {"a":2,"b":[1,2,3]} ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
def toJson(v):
    if it is a string, number, boolean or null
        convert it directly
    if an array or object, convert each item and
        join with commas inside brackets or
        braces (keys in Object.keys order)
```

TIME	SPACE
O(n)	O(n)

NOTE

Not slow: each value is visited once. The recursion depth and the string pieces use O(depth + output) memory; joining arrays of parts avoids repeated concatenation.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Recursive JSON serializer — O(n). Walk-through:"

`stringify(val):`

If null: return 'null'.

If boolean: return 'true' or 'false'.

If number: return String(val).

If string: return "" + val + "". <- wrap in double quotes

If array: '[' + elements.map(stringify).join(',') + ']'.

If object: '{' + entries.map(([k,v]) => ""+k+"":'+stringify(v)).join(',') + '}'.

WHY recursive: JSON is a recursive structure — values can be nested objects/arrays.

WHY quote strings but not numbers/bools: JSON syntax distinguishes types.

Edge: don't include undefined values in objects (JSON.stringify behavior).

Time: O(n) where n = total nodes. Space: O(depth) recursion.

PHASE 4 CLEAN CODE

Clean recursive type dispatch with match-style elif chain."

dict

```
var jsonStringify = function(object) {
    if (object === null) return 'null';
    const type = typeof object;
    if (type === 'number' || type === 'boolean') return St
ring(object);
    if (type === 'string') {
        const escapes = { '"': '\\"', '\\': '\\\\', '\n':
'\\n', '\r': '\\r', '\t': '\\t', '\b': '\\b', '\f': '\\f'
};
        return '"' + object.replace(/[\\"\u0000-\u001f]/g,
            c => escapes[c] || '\\u' + c.charCodeAt(0).toS
tring(16).padStart(4, '0')) + '"';
    }
    if (Array.isArray(object)) return '[' + object.map(v =
> jsonStringify(v)).join(',') + ']';
    return '{' + Object.keys(object).map(k => jsonStringif
y(k) + ':' + jsonStringify(object[k])).join(',') + '}';
};
```

PHASE 5 TEST & DEBUG

"Let me trace jsonStringify({key:'abc',value:123})."

JavaScript JSON.stringify re-implementation (no null/undefined handling)

Input: {key:'abc', value:123}

typeof: 'object' -> not array, not null

Iterate keys: 'key' -> stringify('abc')->'"abc"'; 'value' -> stringify(123)->'123'

Build: '{"key":"abc","value":123}' ok

Trace [1,'abc',null]: typeof -> 'object', Array.isArray -> true

-> '[1,"abc",null]' ok

WHY recursive: objects and arrays can be nested; each type handled separately.

Base cases: null->'null', string->quote, number/boolean->String().

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(d)**
one visit per node	depth

Follow-up: circular reference detection — track visited with a Set.

Follow-up: pretty-print — add indentation parameter.

MEDIUM | JavaScript | LeetCode #2636 | 5 of 7 | 5/7

Promise Pool

QUESTION

Given an array of asynchronous functions functions and a pool limit n, return an asynchronous function promisePool. It should return a promise that resolves when all the input functions resolve.

Pool limit is defined as the maximum number promises that can be pending at once. promisePool should begin execution of as many functions as possible and continue executing new functions when old promises resolve. promisePool should execute functions[i] then functions[i + 1] then functions[i + 2], etc. When the last promise resolves, promisePool should also resolve.

For example, if n = 1, promisePool will execute one function at a time in series. However, if n = 2, it first executes two functions. When either of the two functions resolve, a 3rd function should be executed (if available), and so on until there are no functions left to execute.

You can assume all functions never reject. It is acceptable for promisePool to return a promise that resolves any value.

Example 1

Input	`functions = [ () => new Promise(res => setTimeout(res, 300)), () => new Promise(res => setTimeout(res, 400)), () => new Promise(res => setTimeout(res, 200)) ] n = 2`
Output	`[[300,400,500],500]`
Explanation	*Three functions are passed in. They sleep for 300ms, 400ms, and 200ms respectively. They resolve at 300ms, 400ms, and 500ms respectively. The returned promise resolves at 500ms.* *At t=0, the first 2 functions are executed. The pool size limit of 2 is reached.* *At t=300, the 1st function resolves, and the 3rd function is executed. Pool size is 2.* *At t=400, the 2nd function resolves. There is nothing left to execute. Pool size is 1.* *At t=500, the 3rd function resolves. Pool size is zero so the returned promise also resolves.*

QUESTION

Example 2

Input	`functions = [ () => new Promise(res => setTimeout(res, 300)), () => new Promise(res => setTimeout(res, 400)), () => new Promise(res => setTimeout(res, 200)) ] n = 5`
Output	`[[300,400,200],400]`
Explanation	*The three input promises resolve at 300ms, 400ms, and 200ms respectively. The returned promise resolves at 400ms.* *At t=0, all 3 functions are executed. The pool limit of 5 is never met.* *At t=200, the 3rd function resolves. Pool size is 2.* *At t=300, the 1st function resolved. Pool size is 1.* *At t=400, the 2nd function resolves. Pool size is 0, so the returned promise also resolves.*

Example 3

Input	`functions = [ () => new Promise(res => setTimeout(res, 300)), () => new Promise(res => setTimeout(res, 400)), () => new Promise(res => setTimeout(res, 200)) ] n = 1`
Output	`[[300,700,900],900]`
Explanation	*The three input promises resolve at 300ms, 700ms, and 900ms respectively. The returned promise resolves at 900ms.* *At t=0, the 1st function is executed. Pool size is 1.* *At t=300, the 1st function resolves and the 2nd function is executed. Pool size is 1.* *At t=700, the 2nd function resolves and the 3rd function is executed. Pool size is 1.* *At t=900, the 3rd function resolves. Pool size is 0 so the returned promise resolves.*

CONSTRAINTS

- 0 <= functions.length <= 10
- 1 <= n <= 10

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Execute async functions with at most n concurrent at any time.

Return a promise resolving when all functions complete. Right?"

"A few quick questions: Functions never reject? Yes per constraints."

"Let me walk: 3 functions, n=2: run fn1+fn2 concurrently. As each finishes, start fn3 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
run the functions one at a time: await each before
    starting the next
```

TIME	SPACE
O(sum of durations)	O(1)

WHY IT IS SLOW

Waits for each function to finish before starting the next. A pool starts up to n functions at once and starts the next as soon as any one resolves, so the total time is about the sum of durations divided by n.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Concurrency limiting with a semaphore pattern — O(n/pool size) batches. Walk-through:"

`promisePool(fns, n): at most n promises running concurrently.`

Implementation using a running count or pool:

Use a worker function: while fns remain, pop one, await it, repeat.

Launch n workers simultaneously: await Promise.all(Array(n).fill().map(worker)).

Each worker picks the next available fn from the list and executes it.

When a worker finishes, it immediately picks the next fn — no idle time.

WHY n workers: exactly n promises in-flight at once. Natural concurrency limit.

WHY Promise.all on workers (not on fns): wait for all workers to exhaust the fn list.

Time: O(total work / n) effective parallelism."

Time: O(n) — optimal single-pass solution. Linear time is best achievable.

PHASE 4 CLEAN CODE

Semaphore limits concurrency to n tasks at a time.

```
var promisePool = async function(functions, n) {
    let next = 0;
    async function worker() {
     // each worker keeps taking the next function
        while (next < functions.length) {
            const fn = functions[next++];
            await fn();
        }
    }
    await Promise.all(Array.from({ length: Math.min(n, fun
ctions.length) }, worker));
};
```

PHASE 5 TEST & DEBUG

"Let me trace promisePool([f1,f2,f3,f4], n=2) where each f takes 1s."

Pool of n=2 concurrent promises

Start: launch f1,f2 (pool full)

t=1s: f1 resolves -> launch f3. Active: f2,f3

t=1s: f2 resolves -> launch f4. Active: f3,f4

t=2s: f3,f4 resolve -> done. Total time ~ 2s (not 4s sequential) ok

Implementation: use async/generator or counter

Start n promises; on each resolve, start next in queue until all done.

WHY pool: limits concurrency (e.g., API rate limits); runs n tasks at once optimally.

PHASE 6 COMPLEXITY & FOLLOW-UP

Time optimal. Space O(n) concurrent promises.

Follow-up: cancel on first failure — reject the outer promise on first rejection.

Follow-up: retry failed tasks — wrap fn() in retry loop inside the worker.

MEDIUM | JavaScript | LeetCode #2675 | 6 of 7 | 6/7

Array of Objects to Matrix

QUESTION

Write a function that converts an array of objects arr into a matrix m.

arr is an array of objects or arrays. Each item in the array can be deeply nested with child arrays and child objects. It can also contain numbers, strings, booleans, and null values.

The first row m should be the column names. If there is no nesting, the column names are the unique keys within the objects. If there is nesting, the column names are the respective paths in the object separated by ".".

Each of the remaining rows corresponds to an object in arr. Each value in the matrix corresponds to a value in an object. If a given object doesn't contain a value for a given column, the cell should contain an empty string "".

The columns in the matrix should be in lexographically ascending order.

Example 1

Input

```
arr = [
{"b": 1, "a": 2},
{"b": 3, "a": 4}
]
```

Output

```
[
["a", "b"],
[2, 1],
[4, 3]
]
```

Explanation

There are two unique column names in the two objects: "a" and "b".
"a" corresponds with [2, 4].
"b" coresponds with [1, 3].

Example 2

Input

```
arr = [
{"a": 1, "b": 2},
{"c": 3, "d": 4},
{}
]
```

Output

```
[
["a", "b", "c", "d"],
[1, 2, "", ""],
["", "", 3, 4],
["", "", "", ""]
]
```

Explanation

There are 4 unique column names: "a", "b", "c", "d".
The first object has values associated with "a" and "b".
The second object has values associated with "c" and "d".
The third object has no keys, so it is just a row of empty strings.

QUESTION

Example 3

Input

```
arr = [
{"a": {"b": 1, "c": 2}},
{"a": {"b": 3, "d": 4}}
]
```

Output

```
[
["a.b", "a.c", "a.d"],
[1, 2, ""],
[3, "", 4]
]
```

Explanation

In this example, the objects are nested. The keys represent the full path to each value separated by periods.
There are three paths: "a.b", "a.c", "a.d".

Example 4

Input

```
arr = [
[{"a": null}],
[{"b": true}],
[{"c": "x"}]
]
```

Output

```
[
["0.a", "0.b", "0.c"],
[null, "", ""],
["", true, ""],
["", "", "x"]
]
```

Explanation

Arrays are also considered objects with their keys being their indices.
Each array has one element so the keys are "0.a", "0.b", and "0.c".

Example 5

Input

```
arr = [
{},
{},
{},
]
```

Output

```
[
[],
[],
[],
[]
]
```

Explanation

There are no keys so every row is an empty array.

CONSTRAINTS

- arr is a valid JSON array
- 1 <= arr.length <= 1000
- unique keys <= 1000

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Convert array of objects to a matrix. Row 0 = sorted flattened keys (dot notation).

Remaining rows = values for each object, empty string if key missing. Right?"

"A few quick questions: Arrays inside objects use numeric index as key? Yes."

"Let me walk: [{a:1,b:2},{b:3,c:4}] produces header row [a,b,c] and value rows ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for item in items:
    flatten it into path/value pairs
    keep the header as a list and search it for
        each path to find the column
```

TIME	SPACE
O(items * K^2)	O(items * K)

WHY IT IS SLOW

Searches the header list for every key. Collect all paths first into a set, sort them for the header, and look up each column in a hash map from path to index: O(total size + K log K).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

*"DFS flatten each object, collect keys, build matrix — O(n*k). Walk-through:"*

`flatten(obj, prefix='') -> {dot-path-key: value} dict:`

For each key in obj:

If value is object (non-null): recurse with prefix+key+'.'.

Else: flat[prefix+key] = value.

Collect all unique keys across all flattened objects. Sort them.

Build matrix:

Row 0 = sorted keys.

Row i+1 = [str(flattened_obj[k]) if k in flattened_obj else '' for k in keys].

WHY dot-path keys: JSON paths like 'a.b.c' uniquely identify nested values.

WHY sort keys: ensures consistent column ordering.

Time: O(n*k) where k = total nested keys per object. Space: O(n*k).

PHASE 4 CLEAN CODE

Flatten all objects, collect keys into a set O(n*k), sort once, build matrix.

```
var jsonToMatrix = function(arr) {
    const flatten = (obj, prefix, out) => {
        for (const key of Object.keys(obj)) {
            const path = prefix ? prefix + '.' + key : key
;
            const value = obj[key];
            if (value !== null && typeof value === 'object
') flatten(value, path, out);
            else out[path] = value;
        }
        return out;
    };
    const rows = arr.map(item => flatten(item, '', {}));
     // flatten each item once
    const columns = [...new Set(rows.flatMap(row => Object
.keys(row)))].sort();
    return [columns, ...rows.map(row => columns.map(c => (
c in row ? row[c] : '')))];
};
```

PHASE 5 TEST & DEBUG

"Let me trace jsonToMatrix([{a:1},{a:2,b:3}])."

Step 1 — collect all keys (DFS/flat): obj1->{'a'}; obj2->{'a','b'}. Union -> columns=['a','b'] sorted.

Step 2 — build rows: obj1: row=[obj1.a=1, obj1.b='(missing)] -> [1,'']

obj2: row=[obj2.a=2, obj2.b=3] -> [2,3]

Step 3 — prepend header: [['a','b'],[1,''],[2,3]] ok

Nested trace [{a:{b:1}},{a:{c:2}}]: keys=['a.b','a.c'] (dot notation).

obj1: a.b=1, a.c='' -> [1,'']. obj2: a.b='', a.c=2 -> ['',2].

Result: [['a.b','a.c'],[1,''],['',2]] ok

WHY dot notation: flattens nested keys to CSV-friendly format for spreadsheet export.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n*k*d)	O(n*k) matrix

Follow-up: handle arrays with bracket notation [0],[1].

Follow-up: reverse — matrix to array of objects via dot-path parsing.

IN YOUR APP | Expose | Editorial

MEDIUM | JavaScript | LeetCode #2700 | 7 of 7 | 7/7

Differences Between Two Objects

QUESTION

Write a function that accepts two deeply nested objects or arrays obj1 and obj2 and returns a new object representing their differences.

The function should compare the properties of the two objects and identify any changes. The returned object should only contains keys where the value is different from obj1 to obj2.

For each changed key, the value should be represented as an array [obj1 value, obj2 value]. Keys that exist in one object but not in the other should not be included in the returned object. The end result should be a deeply nested object where each leaf value is a difference array.

When comparing two arrays, the indices of the arrays are considered to be their keys.

You may assume that both objects are the output of JSON.parse.

Example 1:

```
Input:

obj1 = {}
obj2 = {
  "a": 1,
  "b": 2
}

Output:
 {}

Explanation:
 There were no modifications made to obj1. New keys "a" and "b" appear in obj2, but keys that are added or removed should be ignored.
```

Example 2:

QUESTION

```
Input:

obj1 = {
  "a": 1,
  "v": 3,
  "x": [],
  "z": {
    "a": null
  }
}
obj2 = {
  "a": 2,
  "v": 4,
  "x": [],
  "z": {
    "a": 2
  }
}

Output:

{
  "a": [1, 2],
  "v": [3, 4],
  "z": {
    "a": [null, 2]
  }
}

Explanation:
 The keys "a", "v", and "z" all had changes applied. "a" was changed from 1 to 2. "v" was changed from 3 to 4. "z" had a change applied to a child object. "z.a" was changed from null to 2.
```

Example 3:

```
Input:

obj1 = {
  "a": 5,
  "v": 6,
  "z": [1, 2, 4, [2, 5, 7]]
}
obj2 = {
  "a": 5,
  "v": 7,
  "z": [1, 2, 3, [1]]
}

Output:

{
  "v": [6, 7],
  "z": {
    "2": [4, 3],
    "3": {
      "0": [2, 1]
    }
  }
}

Explanation:
 In obj1 and obj2, the keys "v" and "z" have different assigned values. "a" is ignored because the value is unchanged. In the key "z", there is a nested array. Arrays are treated like objects where the indices are keys. There were two alterations to the the array: z[2] and z[3][0]. z[0] and z[1] were unchanged and thus not included. z[3][1] and z[3][2] were removed and thus not included.
```

QUESTION

Example 4:

```
Input:

obj1 = {
  "a": {"b": 1},
}
obj2 = {
  "a": [5],
}

Output:

{
  "a": [{"b": 1}, [5]]
}

Explanation:
 The key "a" exists in both objects. Since the two as
sociated values have different types, they are placed
 in the difference array.
```

Example 5:

```
Input:

obj1 = {
  "a": [1, 2, {}],
  "b": false
}
obj2 = {
  "b": false,
  "a": [1, 2, {}]
}

Output:

{}

Explanation:
 Apart from a different ordering of keys, the two obj
ects are identical so an empty object is returned.
```

CONSTRAINTS

- obj1 and obj2 are valid JSON objects or arrays
- 2 <= JSON.stringify(obj1).length <= 10^4
- 2 <= JSON.stringify(obj2).length <= 10^4

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Find where two JSON values differ. Keys only in ONE object are excluded.

Leaf differences: [obj1_val, obj2_val]. Nested diffs form nested result objects. Right?"

"A few quick questions: Arrays compared by index? Yes — treated like objects."

"Let me walk: diff({a:1,b:2},{a:1,b:3}) gives {b:[2,3]} ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
flatten both objects into maps from key path to value
compare the two maps and report the paths whose
    values differ
```

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

Linear, but flattens everything into extra maps. A direct recursive comparison that returns only the differing parts (and skips equal subtrees) uses O(depth) extra space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Deep object diff: collect changed/added/deleted keys recursively — O(n). Walk-through:"

`diff(obj1, obj2):`

`result = {}.`

For each key in union(keys(obj1), keys(obj2)):

`v1 = obj1[key] (or undefined). v2 = obj2[key] (or undefined).`

If both are plain objects (non-null, non-array): result[key] = diff(v1, v2) if non-empty.

Elif v1 !== v2: result[key] = [v1, v2]. <- changed, added, or deleted

(If v1 === v2: skip — no difference)

Return result.

WHY recurse only on plain objects: arrays and primitives are compared by value, not recursed.

WHY [v1, v2] notation: shows before/after. [undefined, v2] = added. [v1, undefined] = deleted.

Time: O(n) total keys. Space: O(n).

PHASE 4 CLEAN CODE

Single-pass recursive diff — only descend into dicts, treat arrays as leaves.

```
var objDiff = function(obj1, obj2) {
    const isObj = v => v !== null && typeof v === 'object'
;
    if (!isObj(obj1) || !isObj(obj2)) return obj1 === obj2
 ? {} : [obj1, obj2];
    if (Array.isArray(obj1) !== Array.isArray(obj2)) retur
n [obj1, obj2];
    const diff = {};
    for (const key of Object.keys(obj1)) {
        if (!(key in obj2)) continue;
     // keys present in only one object are ignored
        const d = objDiff(obj1[key], obj2[key]);
        if (!Array.isArray(d) && Object.keys(d).length ===
 0) continue;       // nothing changed below this key
        diff[key] = d;
    }
```

```
    return diff;
};
```

PHASE 5 TEST & DEBUG

"Let me trace objDiff({a:1,b:2}, {a:2,b:2})."

Compare two objects recursively:

Key 'a': obj1[a]=1 != obj2[a]=2 -> include {a:{modified:{from:1,to:2}}}?

Actually problem: return object showing what changed

Key 'a': 1->2 -> result.a={__old:1,__new:2}? Depends on spec

Key 'b': 2==2 -> no change, omit

Result: {a:{__old:1,__new:2}} ok (only differing keys shown)

WHY recursion: nested objects need deep comparison; primitives compared directly.

Arrays: compare index by index; missing indices shown as added/removed.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(d)** depth

Follow-up: include keys only in one object as [value, undefined].

Follow-up: JSON Deep Equal (#2628) — same traversal but returns bool.